

Competitive Programming

Section 74 Editorial: Geometric Sweeps And Proximity

Competitive Programming Course

Table of contents

1	How to use this editorial	3
2	A. Orthogonal Crossings	4
2.1	How to find it	4
2.2	Derivation and correctness	4
2.3	Failure-focused tests	4
2.4	C++	4
2.5	Python	6
3	B. Rectangle Union	8
3.1	How to find it	8
3.2	Derivation and correctness	8
3.3	Failure-focused tests	8
3.4	C++	8
3.5	Python	10
4	C. Closest Pair	12
4.1	How to find it	12
4.2	Derivation and correctness	12
4.3	Failure-focused tests	12
4.4	C++	12
4.5	Python	13
5	D. Rectangle Union Perimeter	15
5.1	Why covered length is insufficient	15
5.2	Sweep accounting	15
5.3	Correctness proof	15
5.4	Complexity, mistakes, and tests	16
5.5	C++	16
5.6	Python	18
6	Stress testing and carry-forward	21

1 How to use this editorial

Recover the geometric invariant and degeneracy policy before comparing code.

2 A. Orthogonal Crossings

2.1 How to find it

Sweeping by x turns every horizontal into an active interval in time and every vertical into a y -range query. Write the endpoint convention before selecting the add/query/remove order.

2.2 Derivation and correctness

At sweep coordinate x , the Fenwick tree contains exactly horizontals whose closed x -range contains x because adds precede queries and removals follow them. A vertical query sums exactly active y -values in its closed interval. Total time is $O((H+V) \log(H+V))$.

2.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

2.4 C++

```
#include <bits/stdc++.h>
using namespace std;
struct BIT {
    vector<long long> t;
    BIT(int n) : t(n + 1) {}
    void add(int i, long long v) {
        for (++i; i < (int)t.size(); i += i & -i)
            t[i] += v;
    }
    long long sum(int i) {
        long long s = 0;
        for (; i; i -= i & -i)
            s += t[i];
        return s;
    }
    long long range(int l, int r) { return sum(r) - sum(l); }
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int h, v;
if (!(cin >> h >> v))
    return 0;
struct E {
    long long x;
    int type;
    long long y1, y2, w;
};
vector<E> e;
vector<long long> ys;
for (int i = 0; i < h; ++i) {
    long long x1, x2, y, w = 1;
    cin >> x1 >> x2 >> y;
    if (x1 > x2)
        swap(x1, x2);
    e.push_back({x1, 0, y, y, w});
    e.push_back({x2, 2, y, y, -w});
    ys.push_back(y);
}
for (int i = 0; i < v; ++i) {
    long long x, y1, y2, w = 1;
    cin >> x >> y1 >> y2;
    if (y1 > y2)
        swap(y1, y2);
    e.push_back({x, 1, y1, y2, w});
    ys.push_back(y1);
    ys.push_back(y2);
}
sort(ys.begin(), ys.end());
ys.erase(unique(ys.begin(), ys.end()), ys.end());
sort(e.begin(), e.end(),
      [](E a, E b) { return tie(a.x, a.type) < tie(b.x, b.type); });
BIT bit(ys.size());
long long ans = 0;
for (auto z : e) {
    if (z.type != 1)
        bit.add(lower_bound(ys.begin(), ys.end(), z.y1) - ys.begin(), z.w);
    else {
        int l = lower_bound(ys.begin(), ys.end(), z.y1) - ys.begin(),
            r = upper_bound(ys.begin(), ys.end(), z.y2) - ys.begin();
        ans += z.w * bit.range(l, r);
    }
}
cout << ans << '\n';
}

```

2.5 Python

```
import bisect, sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    h, v = next(it), next(it)
    events = []
    ys = []
    for _ in range(h):
        x1, x2, y = next(it), next(it), next(it)
        x1, x2 = sorted((x1, x2))
        events.extend(((x1, 0, y, y, 1), (x2, 2, y, y, -1)))
        ys.append(y)
    for _ in range(v):
        x, y1, y2 = next(it), next(it), next(it)
        y1, y2 = sorted((y1, y2))
        events.append((x, 1, y1, y2, 1))
        ys.extend((y1, y2))
    ys = sorted(set(ys))
    bit = [0] * (len(ys) + 1)

    def add(i, x):
        i += 1
        while i < len(bit):
            bit[i] += x
            i += i & -i

    def pref(i):
        s = 0
        while i:
            s += bit[i]
            i -= i & -i
        return s

    ans = 0
    for _, kind, y1, y2, w in sorted(events):
        if kind != 1:
            add(bisect.bisect_left(ys, y1), w)
        else:
            ans += w * (
                pref(bisect.bisect_right(ys, y2))
                - pref(bisect.bisect_left(ys, y1))
            )
    print(ans)
```

```
if __name__ == "__main__":  
    main()
```

3 B. Rectangle Union

3.1 How to find it

Do not attempt inclusion-exclusion over rectangles. Sweep between consecutive vertical sides; the only changing quantity is the one-dimensional union length on y.

3.2 Derivation and correctness

The active rectangle set changes only at vertical sides. Between event x-values, covered y-length is constant, so multiplying it by the width is exact. The cover-count segment tree stores the union length of active y-intervals. Complexity is $O(n \log n)$.

3.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

3.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct E {
    ll x, y1, y2;
    int d;
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    vector<E> e;
    vector<ll> ys;
    for (int i = 0; i < n; ++i) {
        ll x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        if (x1 > x2)
            swap(x1, x2);
```

```

    if (y1 > y2)
        swap(y1, y2);
    if (x1 == x2 || y1 == y2)
        continue;
    e.push_back({x1, y1, y2, 1});
    e.push_back({x2, y1, y2, -1});
    ys.push_back(y1);
    ys.push_back(y2);
}
if (e.empty()) {
    cout << 0 << '\n';
    return 0;
}
sort(ys.begin(), ys.end());
ys.erase(unique(ys.begin(), ys.end()), ys.end());
sort(e.begin(), e.end(), [](E a, E b) { return a.x < b.x; });
int m = ys.size() - 1;
vector<int> cover(4 * m);
vector<ll> len(4 * m);
function<void(int, int, int, int, int, int)> upd =
    [&](int x, int l, int r, int ql, int qr, int d) {
        if (qr <= l || r <= ql)
            return;
        if (ql <= l && r <= qr)
            cover[x] += d;
        else {
            int mid = (l + r) / 2;
            upd(2 * x, l, mid, ql, qr, d);
            upd(2 * x + 1, mid, r, ql, qr, d);
        }
        if (cover[x])
            len[x] = ys[r] - ys[l];
        else
            len[x] = r - l == 1 ? 0 : len[2 * x] + len[2 * x + 1];
    };
ll ans = 0, last = e[0].x;
for (int i = 0; i < (int)e.size(); i) {
    ll x = e[i].x;
    ans += (x - last) * len[1];
    while (i < (int)e.size() && e[i].x == x) {
        int l = lower_bound(ys.begin(), ys.end(), e[i].y1) - ys.begin(),
            r = lower_bound(ys.begin(), ys.end(), e[i].y2) - ys.begin();
        upd(1, 0, m, l, r, e[i].d);
        ++i;
    }
    last = x;
}
cout << ans << '\n';
}

```

3.5 Python

```
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    events = []
    ys = []
    for _ in range(n):
        x1, y1, x2, y2 = next(it), next(it), next(it), next(it)
        x1, x2 = sorted((x1, x2))
        y1, y2 = sorted((y1, y2))
        if x1 != x2 and y1 != y2:
            events.extend(((x1, y1, y2, 1), (x2, y1, y2, -1)))
            ys.extend((y1, y2))
    if not events:
        print(0)
        return
    ys = sorted(set(ys))
    index = {y: i for i, y in enumerate(ys)}
    m = len(ys) - 1
    cover = [0] * (4 * m)
    length = [0] * (4 * m)

    def update(x, l, r, ql, qr, d):
        if qr <= l or r <= ql:
            return
        if ql <= l and r <= qr:
            cover[x] += d
        else:
            mid = (l + r) // 2
            update(2 * x, l, mid, ql, qr, d)
            update(2 * x + 1, mid, r, ql, qr, d)
        length[x] = (
            ys[r] - ys[l]
            if cover[x]
            else (0 if r - l == 1 else length[2 * x] + length[2 * x + 1])
        )

    events.sort()
    ans = 0
    last = events[0][0]
    i = 0
    while i < len(events):
        x = events[i][0]
```

```
    ans += (x - last) * length[1]
    while i < len(events) and events[i][0] == x:
        _, y1, y2, d = events[i]
        update(1, 0, m, index[y1], index[y2], d)
        i += 1
    last = x
print(ans)

if __name__ == "__main__":
    main()
```

4 C. Closest Pair

4.1 How to find it

The slow pair search becomes useful after splitting by x : recursive answers eliminate everything except a narrow cross-boundary strip. The remaining question is why y -order plus packing makes that strip linear.

4.2 Derivation and correctness

A better cross-half pair must lie inside the width- $\sqrt{d}$ strip. Sorting it by y permits stopping once the y -gap reaches $\sqrt{d}$; planar packing bounds the remaining comparisons per point by a constant. Merging y -orders at every recursion gives $O(n \log n)$.

4.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

4.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = LLONG_MAX;
struct P {
    ll x, y;
};
ll d2(P a, P b) {
    ll x = a.x - b.x, y = a.y - b.y;
    return x * x + y * y;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    vector<P> a(n), tmp(n);
    for (auto &p : a)
```

```

    cin >> p.x >> p.y;
    sort(a.begin(), a.end(),
        [](P p, P q) { return tie(p.x, p.y) < tie(q.x, q.y); });
    function<ll(int, int)> solve = [&](int l, int r) {
        if (r - l <= 3) {
            ll d = INF;
            for (int i = l; i < r; ++i)
                for (int j = i + 1; j < r; ++j)
                    d = min(d, d2(a[i], a[j]));
            sort(a.begin() + l, a.begin() + r, [](P p, P q) { return p.y < q.y; });
            return d;
        }
        int m = (l + r) / 2;
        ll mid = a[m].x, d = min(solve(l, m), solve(m, r));
        merge(a.begin() + l, a.begin() + m, a.begin() + m, a.begin() + r,
            tmp.begin(), [](P p, P q) { return p.y < q.y; });
        copy(tmp.begin(), tmp.begin() + r - l, a.begin() + l);
        vector<P> s;
        for (int i = l; i < r; ++i)
            if ((a[i].x - mid) * (a[i].x - mid) < d) {
                for (int j = (int)s.size() - 1;
                    j >= 0 && (a[i].y - s[j].y) * (a[i].y - s[j].y) < d; --j)
                    d = min(d, d2(a[i], s[j]));
                s.push_back(a[i]);
            }
        return d;
    };
    cout << solve(0, n) << '\n';
}

```

4.5 Python

```

import sys

INF = 10**50

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    points = sorted(zip(data[1::2], data[2::2]))
    n = len(points)

    def solve(p):
        n = len(p)
        if n <= 3:
            d = min(
                (

```

```

        (p[i][0] - p[j][0]) ** 2 + (p[i][1] - p[j][1]) ** 2
        for i in range(n)
        for j in range(i)
    ),
    default=INF,
)
return d, sorted(p, key=lambda x: x[1])
m = n // 2
mid = p[m][0]
dl, l = solve(p[:m])
dr, r = solve(p[m:])
d = min(dl, dr)
merged = []
i = j = 0
while i < len(l) or j < len(r):
    if j == len(r) or i < len(l) and l[i][1] <= r[j][1]:
        merged.append(l[i])
        i += 1
    else:
        merged.append(r[j])
        j += 1
strip = []
for point in merged:
    if (point[0] - mid) ** 2 < d:
        for other in reversed(strip):
            if (point[1] - other[1]) ** 2 >= d:
                break
            d = min(
                d,
                (point[0] - other[0]) ** 2
                + (point[1] - other[1]) ** 2,
            )
        strip.append(point)
return d, merged

print(solve(points)[0])

if __name__ == "__main__":
    main()

```

5 D. Rectangle Union Perimeter

5.1 Why covered length is insufficient

Problem B's covered y-length gives area, but horizontal perimeter depends on how many disjoint covered y-components exist. Each segment-tree node therefore stores:

```
cover count
covered length
number of covered components
whether its left and right boundaries are covered
```

If local cover is positive, the node is one fully covered component. Otherwise merge its children and subtract one component when their touching boundaries are both covered.

5.2 Sweep accounting

Between consecutive x-coordinates, coverage is constant. For strip width dx , add

```
area      += covered_length * dx
perimeter += 2 * components * dx.
```

At an event coordinate, vertical boundary equals the length covered on exactly one side of the sweep. Process all additions before removals and, after each update, add the absolute change in covered length. Additions measure coverage new on the right; subsequent removals measure coverage lost from the left that was not replaced by an addition.

5.3 Correctness proof

Lemma 1. The tree root stores the exact active covered length and number of connected y-components.

Proof. A positively covered node is its complete geometric interval and one component. An uncovered leaf contributes nothing. For an uncovered internal node, child unions are disjoint except possibly at their shared endpoint; their component counts add, merging exactly once when both touching boundaries are covered. Structural induction proves every stored field. QED.

Lemma 2. Strip contributions equal all nonvertical boundary and area in that strip.

Proof. Active coverage is constant inside the open strip. Its Cartesian product with width dx has the stated area. Every covered y-component has one top and one bottom boundary spanning the strip, producing $2 * components * dx$. QED.

Lemma 3. Event contributions equal all vertical union boundary at that x .

Proof. Vertical boundary is the symmetric difference of covered y -sets immediately before and after the batch. Adding intervals first accumulates the measure newly covered after the event. Removing ending intervals afterward accumulates measure covered before but absent after, while portions replaced by additions remain covered and cause no decrease. QED.

The three lemmas partition the union's area and boundary, proving both outputs.

5.4 Complexity, mistakes, and tests

There are $2n$ events and $O(n)$ compressed y -intervals. Sorting costs $O(n \log n)$ and every update costs $O(\log n)$.

Common mistakes include using compressed index counts as length, omitting the child-boundary merge, processing removals before additions at the same x , and adding `2*covered_length` instead of `2*components` per unit width. Test identical rectangles, shared edges, corner contact, nesting, disjoint pieces, and several starts and ends at one x .

5.5 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

struct Event {
    int64 x, y1, y2;
    int delta;
};

struct CoverTree {
    int intervals;
    const vector<int64>& coordinate;
    vector<int> cover, components;
    vector<int64> length;
    vector<char> left_covered, right_covered;

    explicit CoverTree(const vector<int64>& ys)
        : intervals((int)ys.size() - 1), coordinate(ys),
          cover(4 * max(1, intervals)), components(cover.size()),
          length(cover.size()), left_covered(cover.size()),
          right_covered(cover.size()) {}

    void pull(int node, int low, int high) {
        if (cover[node] > 0) {
            length[node] = coordinate[high] - coordinate[low];
            components[node] = 1;
            left_covered[node] = right_covered[node] = true;
        } else if (high - low == 1) {}
    }
};
```

```

        length[node] = 0;
        components[node] = 0;
        left_covered[node] = right_covered[node] = false;
    } else {
        int left = 2 * node;
        int right = left + 1;
        length[node] = length[left] + length[right];
        components[node] = components[left] + components[right]
            - (right_covered[left] && left_covered[right]);
        left_covered[node] = left_covered[left];
        right_covered[node] = right_covered[right];
    }
}

void update(int from, int to, int delta, int node, int low, int high) {
    if (to <= low || high <= from) return;
    if (from <= low && high <= to) {
        cover[node] += delta;
        pull(node, low, high);
        return;
    }
    int middle = (low + high) / 2;
    update(from, to, delta, 2 * node, low, middle);
    update(from, to, delta, 2 * node + 1, middle, high);
    pull(node, low, high);
}

void update(int from, int to, int delta) {
    update(from, to, delta, 1, 0, intervals);
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Event> events;
    vector<int64> ys;
    for (int i = 0; i < n; ++i) {
        int64 x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        events.push_back({x1, y1, y2, +1});
        events.push_back({x2, y1, y2, -1});
        ys.push_back(y1);
        ys.push_back(y2);
    }
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    sort(events.begin(), events.end(), [](const Event& a, const Event& b) {

```

```

        if (a.x != b.x) return a.x < b.x;
        return a.delta > b.delta; // additions before removals
    });

    CoverTree tree(ys);
    int64 area = 0;
    int64 perimeter = 0;
    int64 previous_x = events.front().x;
    int index = 0;
    while (index < (int)events.size()) {
        int64 x = events[index].x;
        int64 width = x - previous_x;
        area += tree.length[1] * width;
        perimeter += int64(2) * tree.components[1] * width;

        while (index < (int)events.size() && events[index].x == x) {
            int from = lower_bound(ys.begin(), ys.end(), events[index].y1)
                - ys.begin();
            int to = lower_bound(ys.begin(), ys.end(), events[index].y2)
                - ys.begin();
            int64 before = tree.length[1];
            tree.update(from, to, events[index].delta);
            perimeter += llabs(tree.length[1] - before);
            ++index;
        }
        previous_x = x;
    }

    cout << area << ' ' << perimeter << '\n';
}

```

5.6 Python

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    events = []
    ys = []
    at = 1
    for _ in range(n):
        x1, y1, x2, y2 = data[at : at + 4]
        at += 4
        events.append((x1, -1, y1, y2)) # additions sort first

```

```

        events.append((x2, +1, y1, y2))
        ys.extend((y1, y2))
    ys = sorted(set(ys))
    position = {value: index for index, value in enumerate(ys)}
    events.sort()

    size = 4 * (len(ys) - 1)
    cover = [0] * size
    length = [0] * size
    components = [0] * size
    left_covered = [False] * size
    right_covered = [False] * size

    def pull(node: int, low: int, high: int) -> None:
        if cover[node] > 0:
            length[node] = ys[high] - ys[low]
            components[node] = 1
            left_covered[node] = right_covered[node] = True
        elif high - low == 1:
            length[node] = components[node] = 0
            left_covered[node] = right_covered[node] = False
        else:
            left = 2 * node
            right = left + 1
            length[node] = length[left] + length[right]
            components[node] = (
                components[left] + components[right]
                - (right_covered[left] and left_covered[right])
            )
            left_covered[node] = left_covered[left]
            right_covered[node] = right_covered[right]

    def update(start: int, end: int, delta: int, node=1, low=0, high=None) -> None:
        if high is None:
            high = len(ys) - 1
        if end <= low or high <= start:
            return
        if start <= low and high <= end:
            cover[node] += delta
            pull(node, low, high)
            return
        middle = (low + high) // 2
        update(start, end, delta, 2 * node, low, middle)
        update(start, end, delta, 2 * node + 1, middle, high)
        pull(node, low, high)

    area = perimeter = 0
    previous_x = events[0][0]
    index = 0
    while index < len(events):
        x = events[index][0]

```

```

width = x - previous_x
area += length[1] * width
perimeter += 2 * components[1] * width
while index < len(events) and events[index][0] == x:
    _, encoded_delta, y1, y2 = events[index]
    before = length[1]
    update(position[y1], position[y2], -encoded_delta)
    perimeter += abs(length[1] - before)
    index += 1
previous_x = x

print(area, perimeter)

if __name__ == "__main__":
    main()

```

6 Stress testing and carry-forward

The D oracle rasterizes small integer rectangles into unit cells. Area is the number of occupied cells and perimeter is the number of cell sides adjacent to an unoccupied cell. The transferable state-design lesson is that covered length alone loses topology; component count plus endpoint flags is the smallest extra boundary summary needed here.